Coding & Solution Implementation Report

Implementation coverage and endpoint contract

Project: AI WeatherWise API   Track: AI-ML & GEN-AI Track

Prepared: 07 October 2026   Team ID: Not provided

Maximum Marks: 5

Team Members & Assigned Roles

Team Member	Role
Jai Bharath A	Team Lead & Lead Backend/API Architect
Mukesh Balaji	Generative AI & Prompt Engineering Engineer
Yadagiri Mani	Database & Data Engineering Specialist
Deepak G	Security, Testing & Reliability Engineer
Naveen	API Integration, Documentation & Demonstration Engineer

Implementation Summary

The supplied project specification describes a Node.js ES6+ asynchronous Express backend using Mongoose, OpenWeatherMap and Gemini 2.5 Flash. The implementation is organized as an MVC-style REST API with service-level fallback behavior.

Technology	Version / approach	Usage
Node.js	ES6+ / async-await	Runtime
Express.js	4.x style REST API	HTTP layer
Mongoose	8.x compatible	MongoDB ODM
@google/generative-ai	Gemini 2.5 Flash	Generative AI
bcryptjs	Salted hashing	Passwords
jsonwebtoken	HMAC-SHA256 JWT	Authentication

Endpoint Implementation Matrix

#	Method	Endpoint	Access	Purpose
1	POST	/api/auth/register	Public	Create user; bcrypt password hashing; unique email validation.
2	POST	/api/auth/login	Public	Validate credentials and issue a 30-day HMAC-SHA256 JWT.
3	GET	/api/auth/profile	Protected	Return the authenticated user's profile.
4	GET	/api/weather/:city	Public	Fetch real-time OpenWeatherMap data with deterministic mock fallback.
5	POST	/api/locations	Protected	Save a favorite city/location with ownership and uniqueness controls.
6	GET	/api/locations	Protected	List locations belonging only to the authenticated user.
7	PUT	/api/locations/:id	Protected	Update location nickname or personal notes.
8	DELETE	/api/locations/:id	Protected	Delete a location after ownership validation.
9	POST	/api/ai/weather-summary	Public	Generate a natural-language weather summary with Gemini 2.5 Flash.
10	POST	/api/ai/weather-recommendation	Public	Generate contextual outfit, commute and hydration recommendations.

Implementation Principles

- Controllers handle HTTP concerns; services handle external API/AI logic.
- Protected routes derive user identity from verified JWTs rather than client-supplied user IDs.
- Weather fallback data is deterministic so testing is reproducible.
- AI fallback logic is local and rule-based, ensuring a useful response during quota or network failures.